

Technische Hochschule Ingolstadt
Fakultät Elektro- und Informationstechnik

Sensor Network Technologies and Sensor Fusion

AdaBoost / Viola Jones
Single Neurons, Gradient Descent

Oct 09, 2025

Welcome to the lecture!

Prof. Dr.-Ing. Björn Giesler

Technische Hochschule Ingolstadt / Fraunhofer IVI

Autonomous, Cooperating Systems

Email: bjoern.giesler@thi.de

- **Please register for the lecture on Moodle!**
  - Link: https://moodle.thi.de/course/view.php?id=10580
  - Key: **SensNT+DF#WS25**
- **News on Moodle:**
  - Slides for the lecture (upcoming lectures marked as WIP and are subject to change!)
  - Formulary, Exercises (Solutions provided after discussion in the lecture)
  - Exercises are optional, one task / sheet per week, but are highly recommended.
    They can be handed in for correction.
  - I will *try* to publish exercises before Wednesday‘s lecture so that initial questions can be discussed on Thursday.
- **Exam: Written exam, 90 minutes**
- **Consultation hours: Please arrange via e-mail to bjoern.giesler@thi.de.**
- **Please also don‘t hesitate to send questions, lecture feedback, change requests, etc.!**

Goal: Obtain 3D objects from video



- **Sensing with Cameras**
  - Gradient Descent
  - Neural Networks

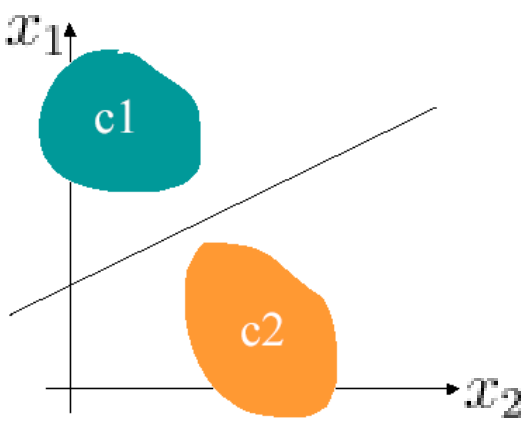
Machine Learning: AdaBoost

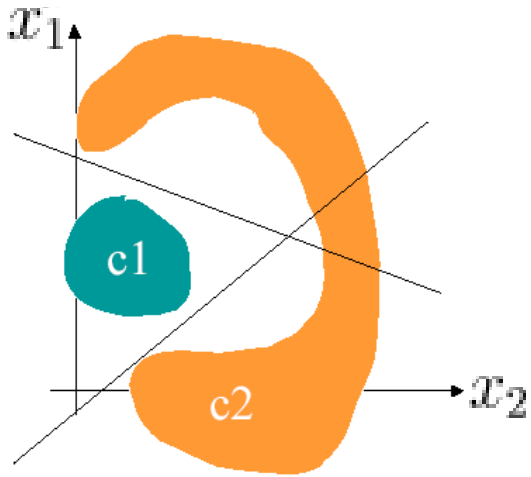
- **Artificial intelligence is *the capability of computational systems to perform tasks typically associated with human intelligence, such as learning, reasoning, problem-solving, perception, and decision-making* (Wikipedia).**
- **That‘s a lot. We will start with the basics.**
- **Machine Learning is focused on teaching software systems to learn from data, without being explicitly programmed.**
- **At its root, it solves a clustering task: Transformation of a *feature vector* into the likelihood of belonging to certain elements of a *class vector.***

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \rightarrow \begin{pmatrix} q_{c_1} \\ q_{c_2} \\ \vdots \\ q_{c_3} \end{pmatrix}$$ **, where** $$\Omega = \begin{pmatrix} c_1 \\ c_2 \\ \vdots \\ c_m \end{pmatrix}$$ **is the vector of classes to distinguish.**

We call Ω the „ontology“ (as it‘s everything the system knows about the world).

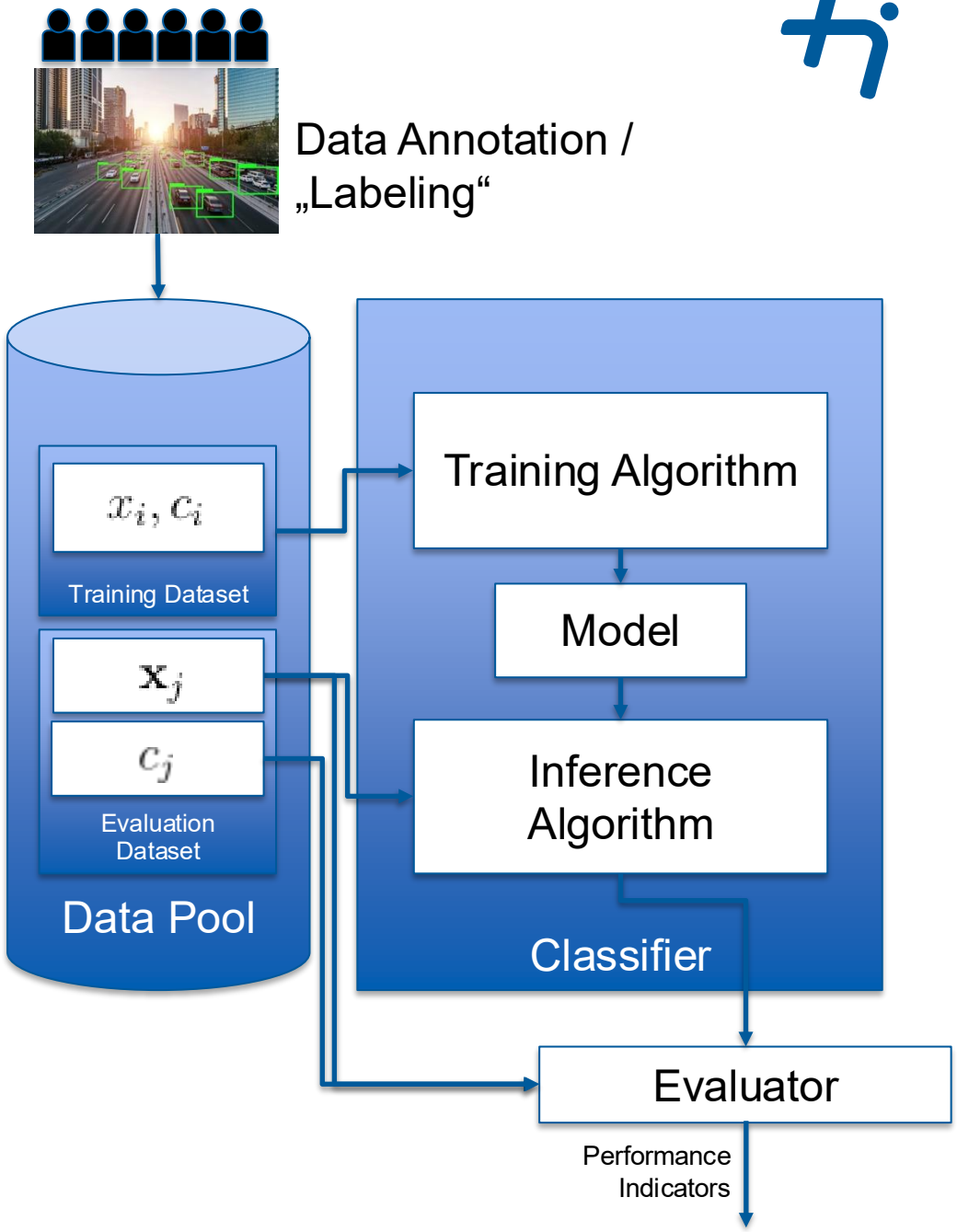
- **We distinguish *learning* (or *training*) and *inference.***
- **Schematically, we can think of the *learning* task as finding separators in an extremely high-dimensional space.**
- **The *inference* task calculates the measure of distance of a data vector to the set of separators.**





- **Classically, we distinguish**
  - Supervised learning: Train a system on manually annotated (e.g. classified) data; the class of each data vector in the training set is initially known.
  - Unsupervised learning: Train a system on un-annotated data, requiring it to derive its own classification.
  - Reinforcement learning: Let the system „try out“ results of subsequent unsupervised training runs, with a quality function or human supervising the quality of the results and guiding the system towards better perfomance.
- **We will examine supervised learning first.**

- **Supervised learning requires manually annotated data.**
- **This data is split into a *training* and an *evaluation* dataset.**
- **For evaluation to be meaningful, both datasets need to be**
  - sufficiently large
  - sufficiently complete
  - disjunct
- **The training algorithm trains the model on the training dataset.**
- **Afterwards, an inference run applies the trained model on the evaluation dataset.**
- **The results of the inference are compared to the labeled ground truth to determine model performance.**



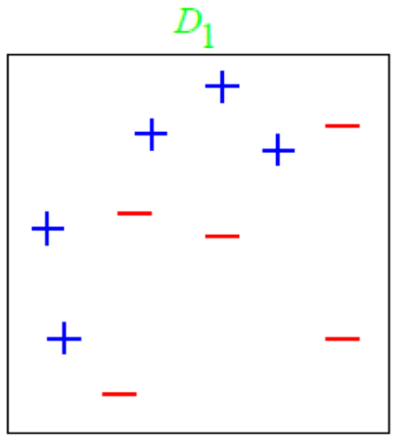
- **Viola & Jones use the Adaptive Boost or AdaBoost algorithm published by Yoav Freund and Robert Schapire in 1999.**
- **AdaBoost is a method of training a boosted classifier, which is a classifier of the form**

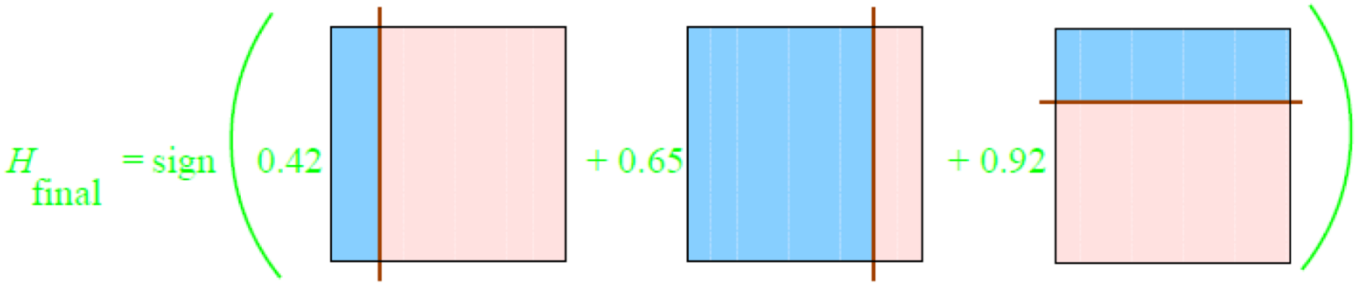
$$F_T(x) = \sum_{t=1}^{T} f_t(x)$$

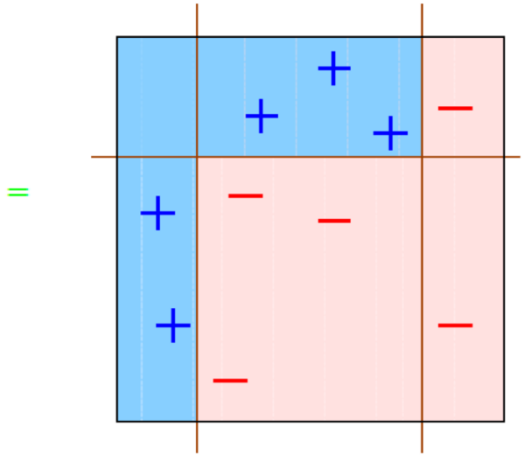
  where each $f_t(x)$ is a weak learner returning an estimate of object class.
- **For the problem of detecting objects of one class (and rejecting all others, i.e. a two-class problem), the sign of $f_t(x)$ represents class and the absolute value represents certainty.**
- **The algorithm iterates through a training set consisting of objects x_i.**
  At each iteration t, a weak learner is selected and assigned a coefficient α_i so that the total training error is minimized:

$$E_t = \sum_i E(F_{t-1}(x_i) + \alpha_t h(x_i))$$

A very simple illustrative example [Ronja Güldenring, Objektklassifizierung mittels AdaBoost-Algorithmus]:

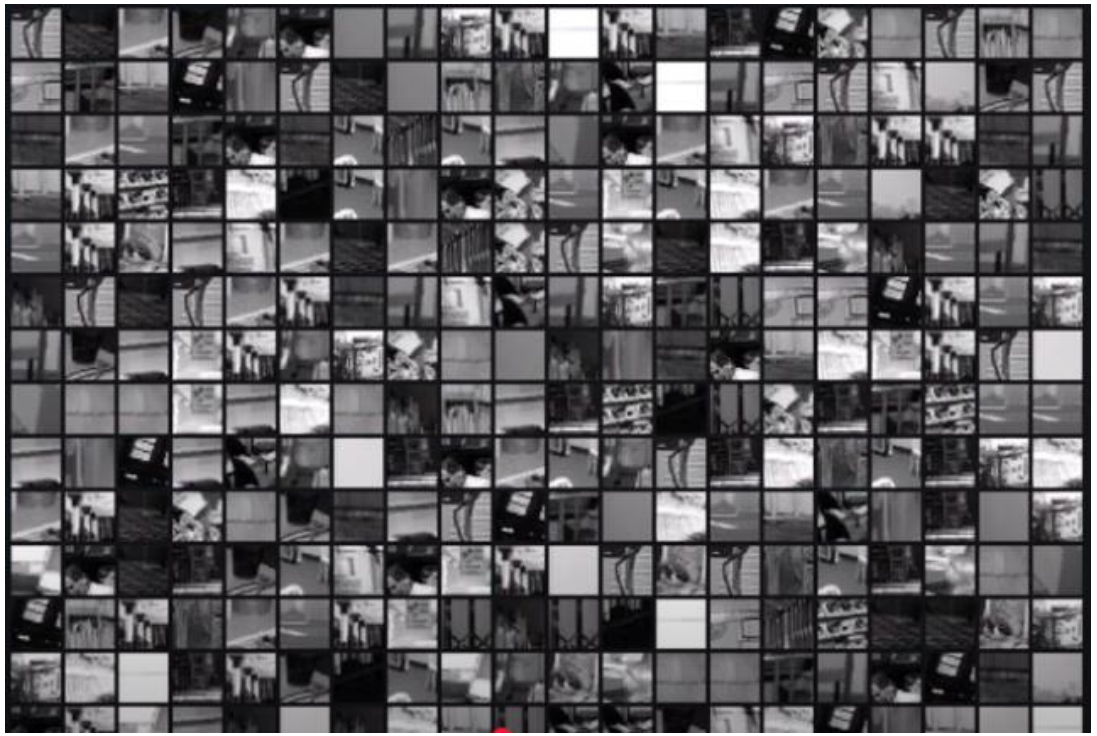








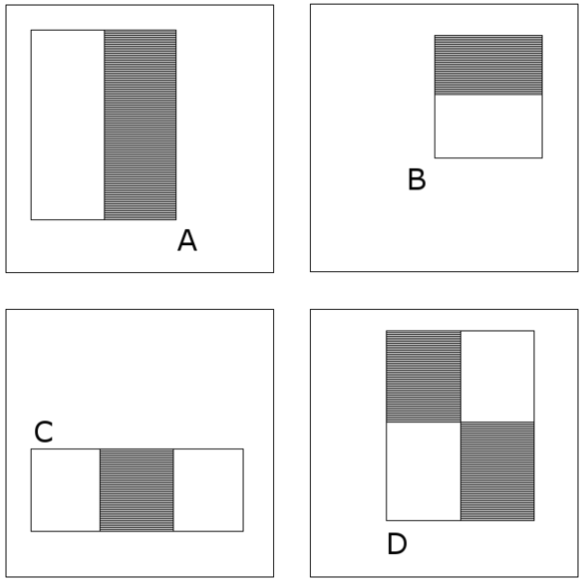
Face Dataset



Rejection Dataset
(randomly picked from arbitrary images)



- **Viola Jones Recognizer: The first „Machine Learning“ Classifier**
- **Described in Paul Viola, Michael Jones: Robust Real-Time Object Detection, 2001**
- **Basic idea:**
  - Objects of different class often have distinct brightness distributions in the greyscale image – e.g. the eye area of a face is often darker than the area directly below, the nose is brighter than the eye area to the left and right.
  - Classification can be achieved by combining different filter masks with different weights.
  - Objects often appear at different scales or sizes in the image, but the basic appearance is similar.
  - Find a combination of elementary scaleable filter masks that sum together to form a strong classifier.
- **Individual filter masks given on the right – result is the sum of pixels “under the black part“ subtracted from the sum of pixels „under the white part“.**



- **The original algorithm uses a pre-computed integral image given by**

$$\mathrm{II}(u,v) = \sum_{i<u} \sum_{j<v} \mathrm{Img}(u,v)$$

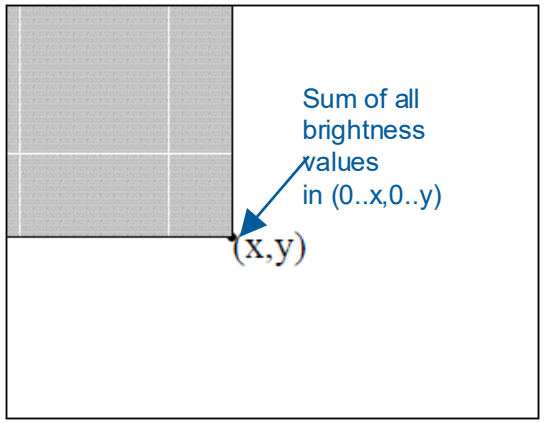
- **This reduces the computation for one region of a weak classifier to four additions:**

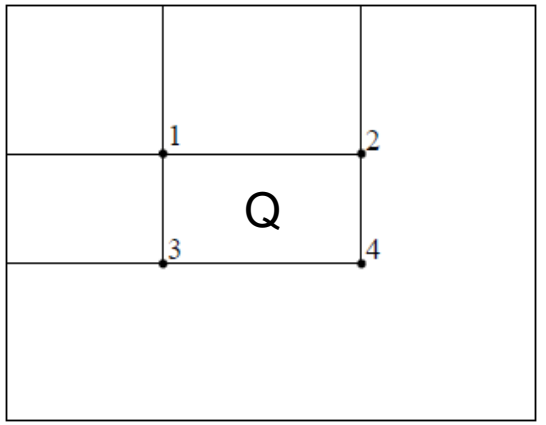
$$\sum Q = II(4) + II(1) - II(2) - II(3)$$

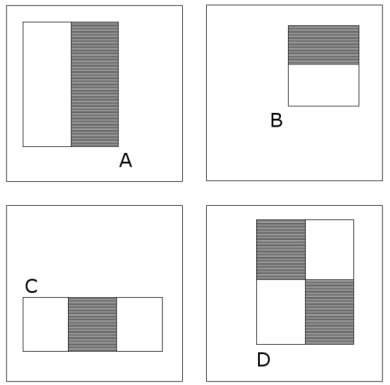
- **This makes the classifier very efficient and runnable in real time even on weak hardware.**

- **The patterns A, B, C, D can be computed at any scale and location in the training image, given a full set of *Weak Learners.***

- **We use the weak learner that best separates the training classes in each step.**









Integral Image Example

1	7	4	2	9
7	2	3	8	2
1	8	7	9	1
3	2	3	1	5
2	9	5	6	6

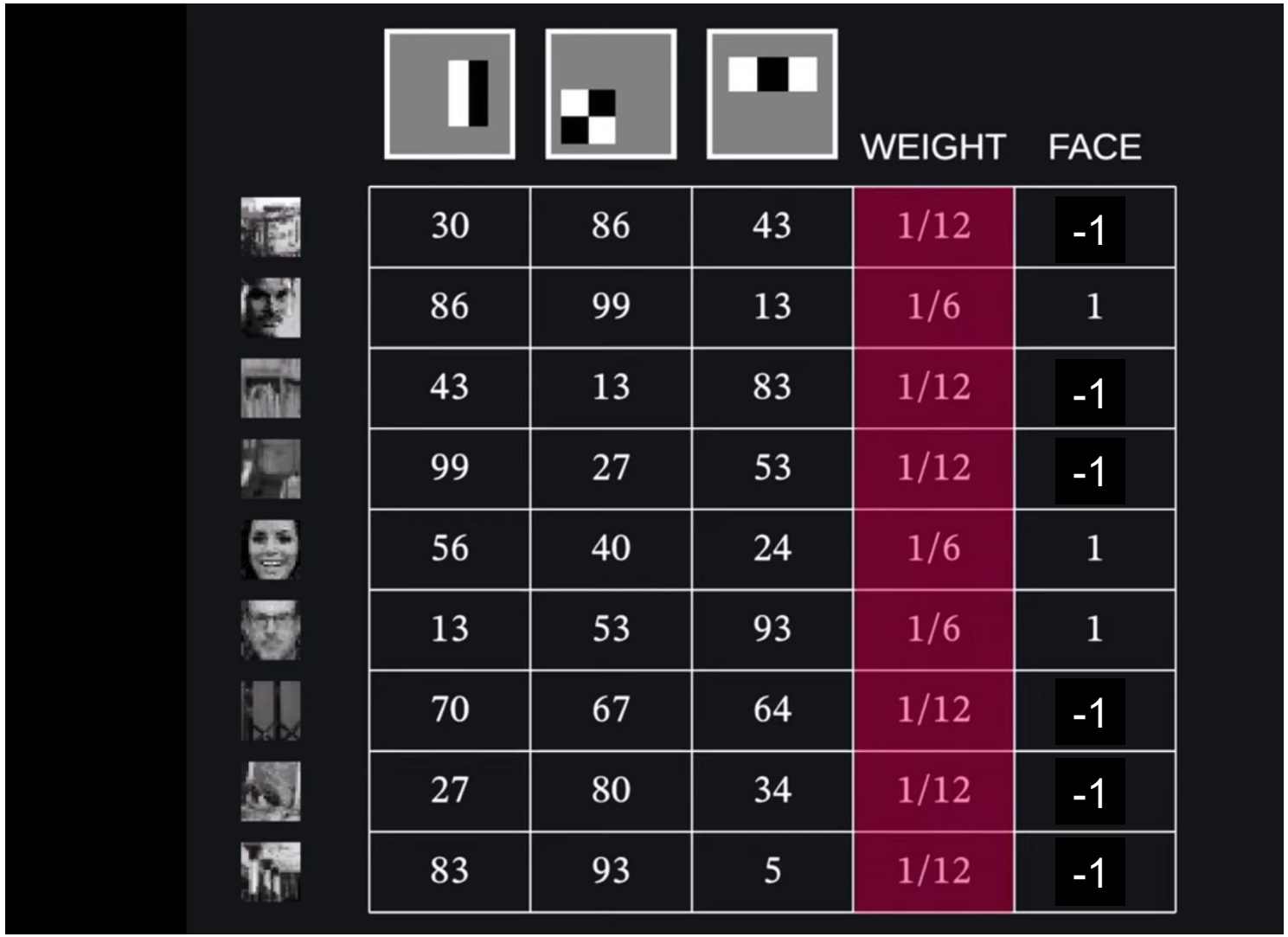
Source Image



1	1+7 =8	8+4 =12	12+2 =14	14+9 =23
1+7 =8	7+7+1 +2 =17	1+7+4 +7+2+ 3=24	34	45
9	26	40	59	71
12	31	48	68	85
14	42	64	90	113

Integral Image

Sum over area:
113+17-45-42 = 43



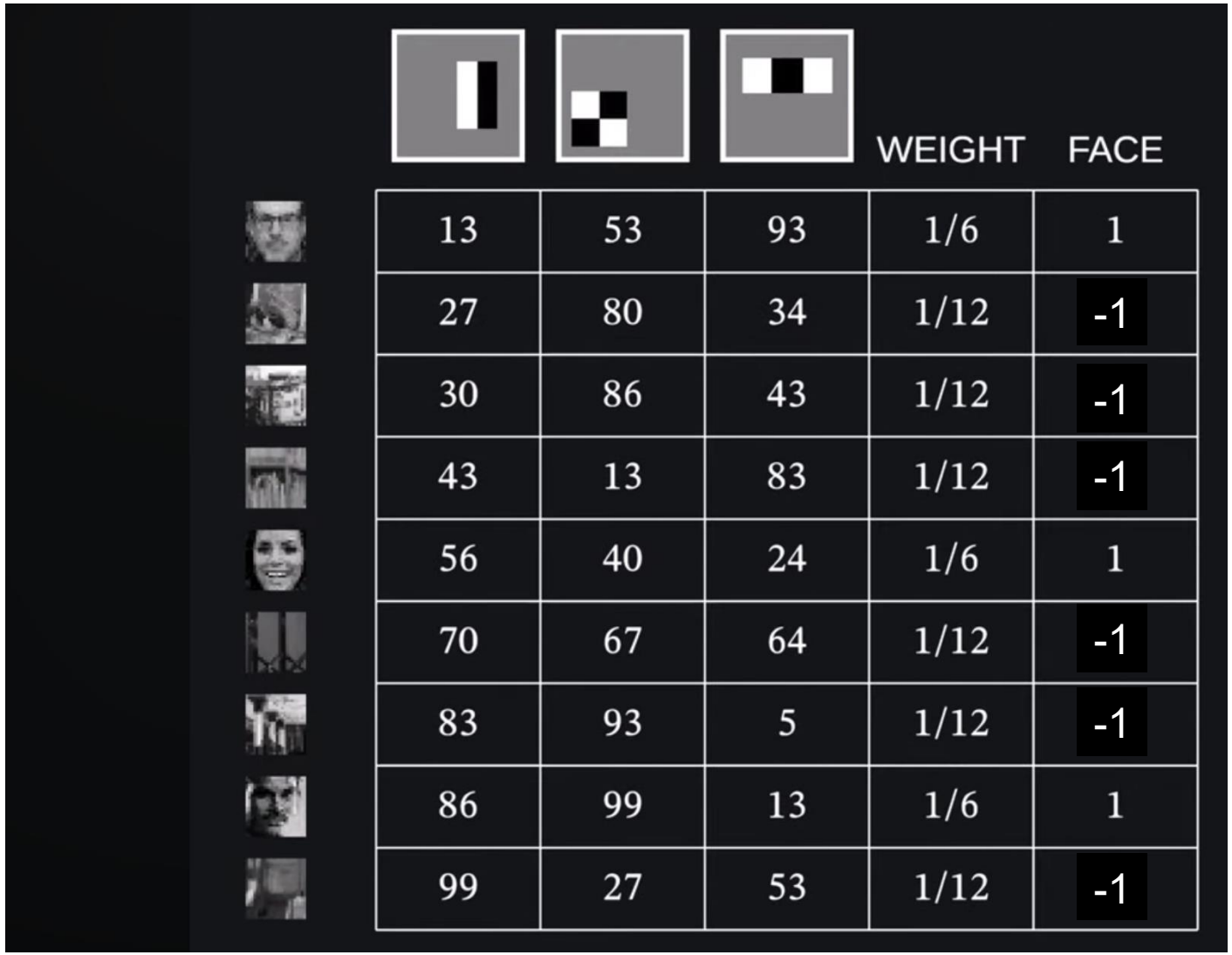
			WEIGHT	FACE
30	86	43	1/12	-1
86	99	13	1/6	1
43	13	83	1/12	-1
99	27	53	1/12	-1
56	40	24	1/6	1
13	53	93	1/6	1
70	67	64	1/12	-1
27	80	34	1/12	-1
83	93	5	1/12	-1

In the first step, assign weights to samples according to

$\frac{1}{2n_{\text{face}}}$ for each face and

$\frac{1}{2n_{\text{rej}}}$ for each rejection sample.

n_{face} and n_{rej} are the number of face and rejection samples, respectively.
The important thing is that all weights in one class add up to ½ so that both classes are weighted equally.



			WEIGHT	FACE
13	53	93	1/6	1
27	80	34	1/12	-1
30	86	43	1/12	-1
43	13	83	1/12	-1
56	40	24	1/6	1
70	67	64	1/12	-1
83	93	5	1/12	-1
86	99	13	1/6	1
99	27	53	1/12	-1

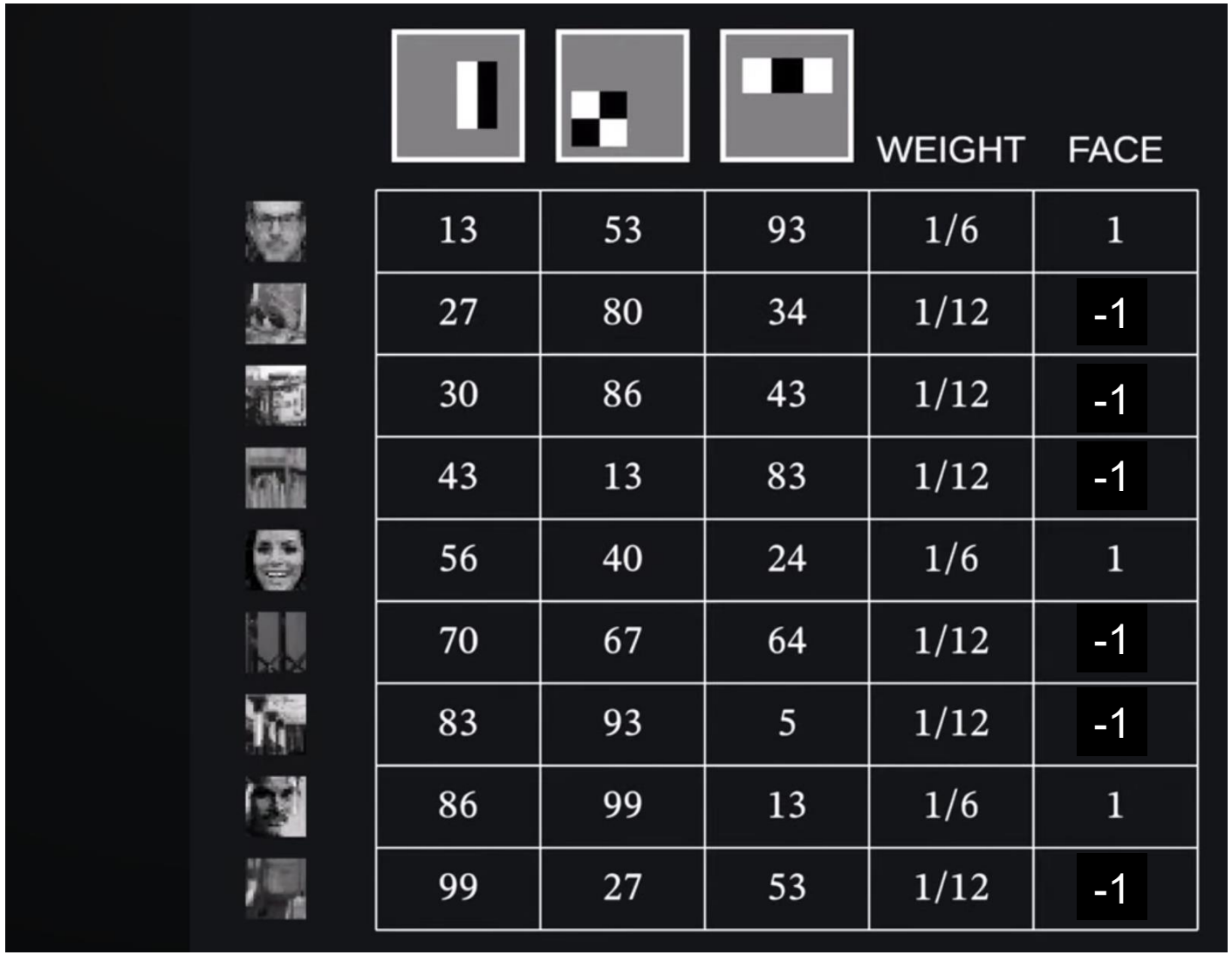
Performance Change	Running Perf Sum
+1/6 = 2/12	2/12
-1/12	1/12
-1/12	0
-1/12	-1/12
+2/12	1/12
-1/12	0
-1/12	-1/12
+2/12	1/12
-1/12	0

In the second step, sort by the feature response of the first feature.

Calculate the error rate, and select the threshold for this feature corresponding to the highest performance (= lowest error). The corresponding line has the minimum response for that feature.

Sensor Networks and Sensor Fusion

Sensing with Cameras



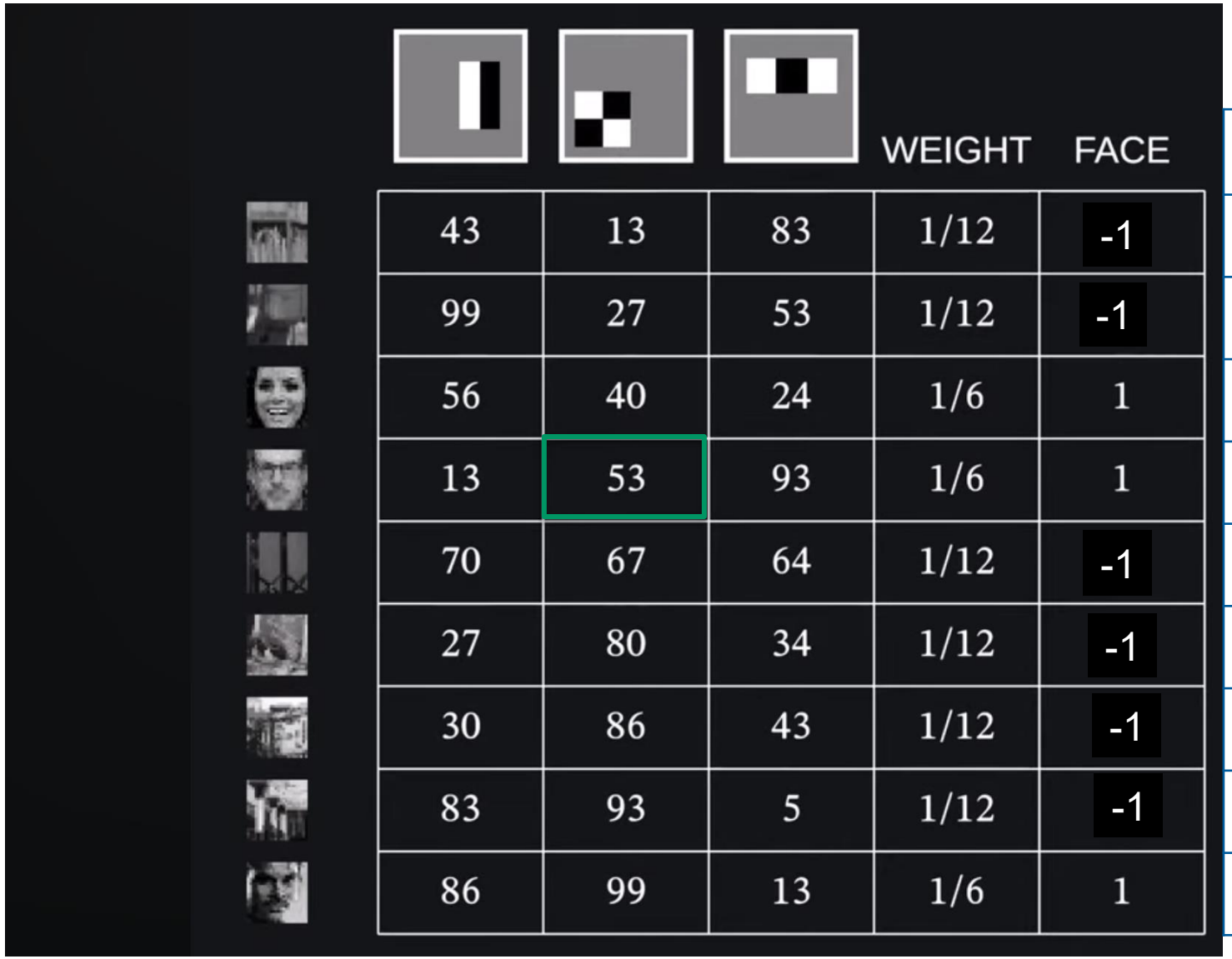
			WEIGHT	FACE
13	53	93	1/6	1
27	80	34	1/12	-1
30	86	43	1/12	-1
43	13	83	1/12	-1
56	40	24	1/6	1
70	67	64	1/12	-1
83	93	5	1/12	-1
86	99	13	1/6	1
99	27	53	1/12	-1

Perf Change	Running Perf Sum
+1/6 = 2/12	2/12
-1/12	1/12
-1/12	0
-1/12	-1/12
+2/12	1/12
-1/12	0
-1/12	-1/12
+2/12	1/12
-1/12	0

In the second step, sort by the feature response of the first feature.

Calculate the error rate, and select the threshold for this feature corresponding to the lowest error. The corresponding line has the minimum response for that feature.

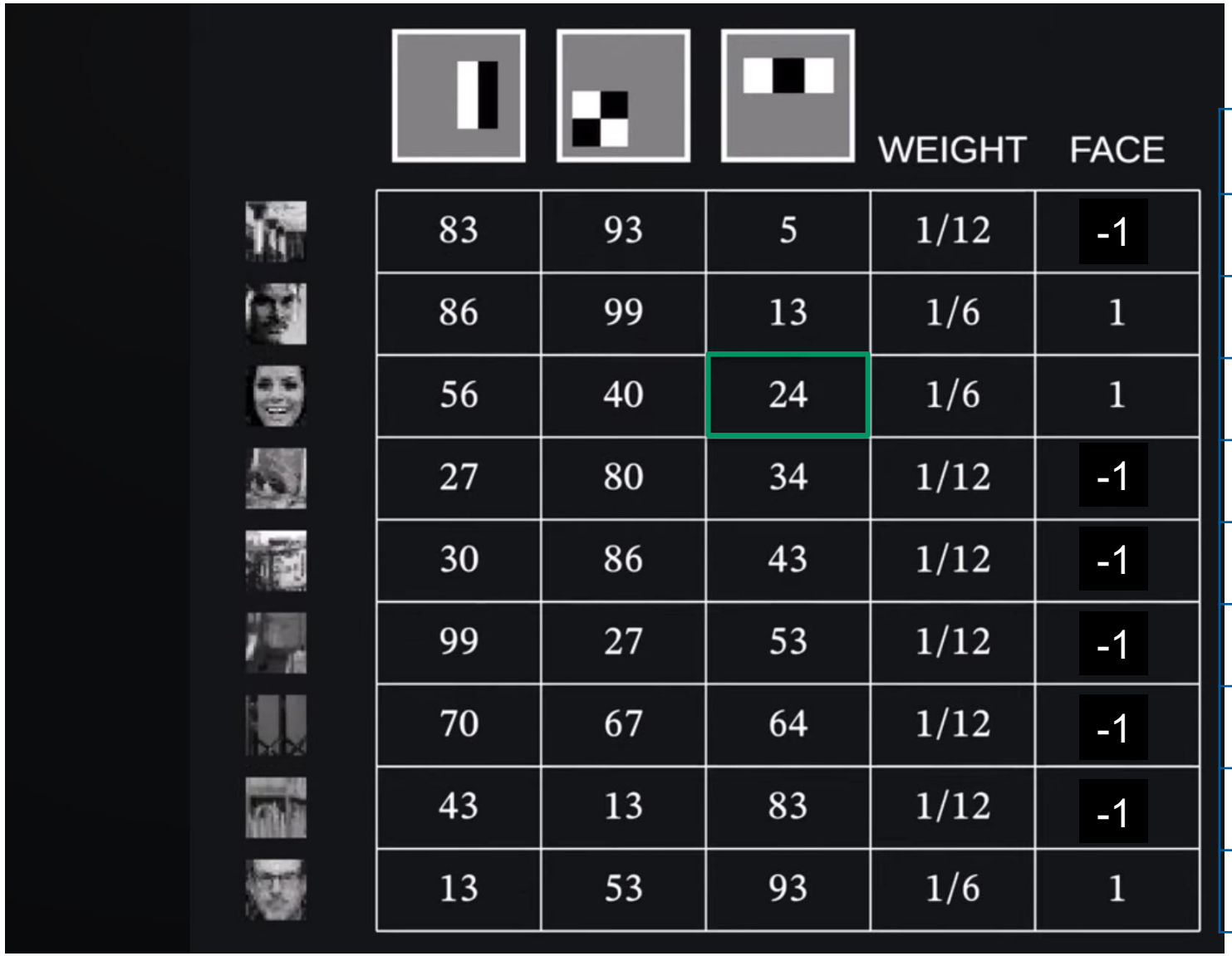
Here, this is the first line, with a minimum response of 13.



			WEIGHT	FACE
43	13	83	1/12	-1
99	27	53	1/12	-1
56	40	24	1/6	1
13	53	93	1/6	1
70	67	64	1/12	-1
27	80	34	1/12	-1
30	86	43	1/12	-1
83	93	5	1/12	-1
86	99	13	1/6	1

Error Change	Running Error Sum
-1/12	-1/12
-1/12	-2/12
+2/12	0
+2/12	2/12
-1/12	1/12
-1/12	0
-1/12	-1/12
-1/12	-2/12
+2/12	0

Repeat for feature 2, sorting by feature response, calculating performance and choosing the threshold corresponding to the highest performance (=smallest error)…

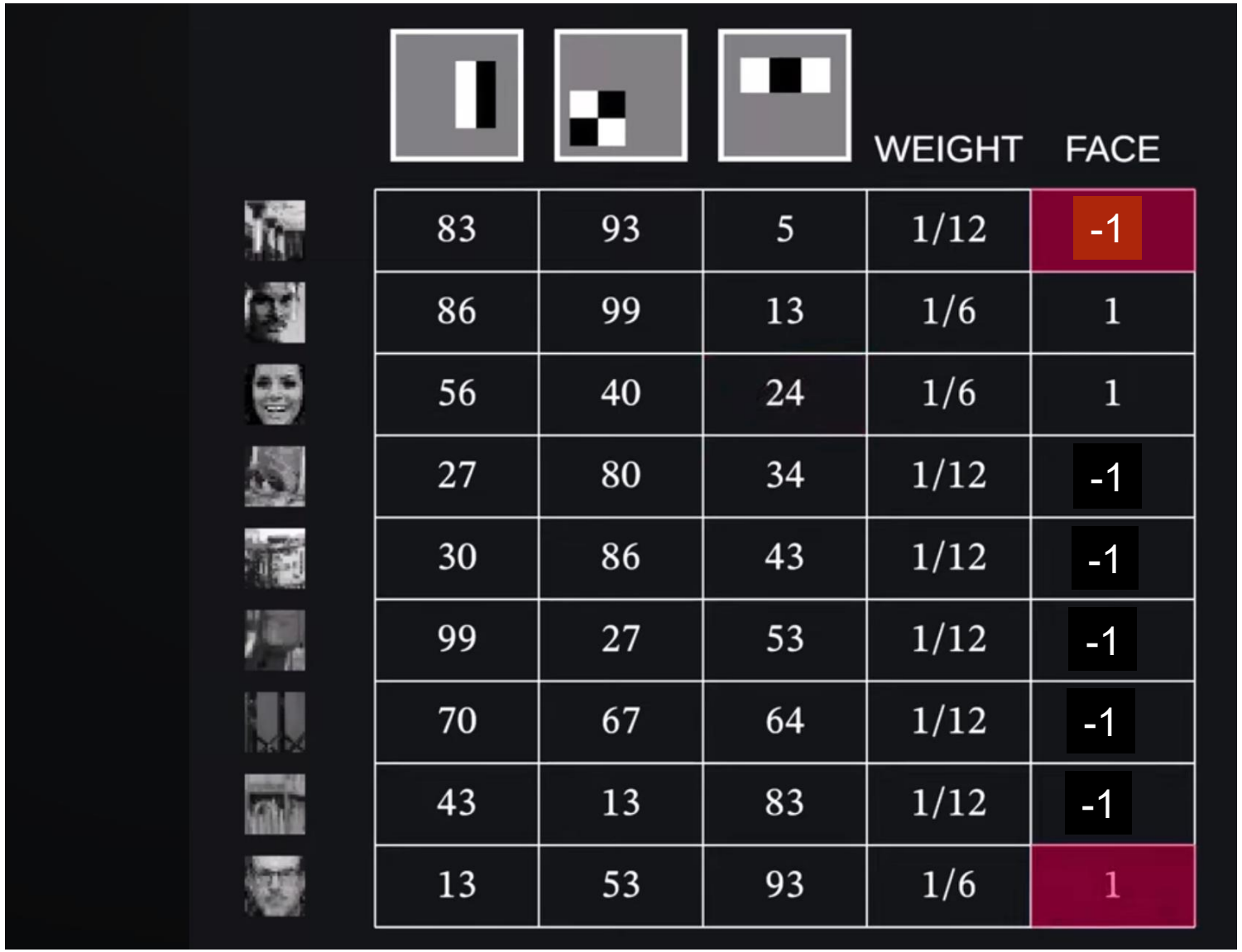


			WEIGHT	FACE
83	93	5	1/12	-1
86	99	13	1/6	1
56	40	24	1/6	1
27	80	34	1/12	-1
30	86	43	1/12	-1
99	27	53	1/12	-1
70	67	64	1/12	-1
43	13	83	1/12	-1
13	53	93	1/6	1

Error Change	Running Error Sum
-1/12	-1/12
+2/12	1/12
+2/12	3/12
-1/12	2/12
-1/12	1/12
-1/12	0
-1/12	-1/12
-1/12	-2/12
+2/12	0

Repeat for feature 3.

Feature 3, with a maximum performance of 3/12, beats features 1 and 2 with 2/12. It will reject everything above a threshold of 24.

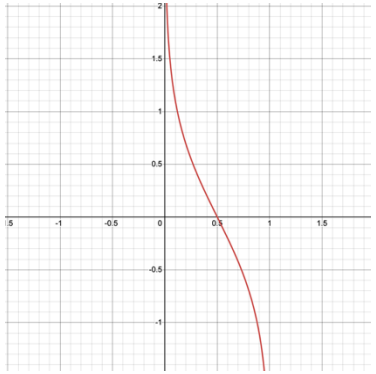


			WEIGHT	FACE
83	93	5	1/12	-1
86	99	13	1/6	1
56	40	24	1/6	1
27	80	34	1/12	-1
30	86	43	1/12	-1
99	27	53	1/12	-1
70	67	64	1/12	-1
43	13	83	1/12	-1
13	53	93	1/6	1

Selecting feature 3 leaves two false classifications in the data set. We need to adapt the weights to make sure these are considered important for the next run.

- **Example of Discrete Two-Class AdaBoost:**
- **With**
  - Samples $(x_1 ... x_n)$ and desired outputs $(y_1 ... y_n), y \in \{-1, 1\}$
  - Initial weights $w_{n,1} = \begin{cases} 1/2n_{class} & \text{if} y_n = 1 \text{, i.e. the sample belongs to the class we want to find} \\ 1/2n_{rej} & \text{if} y_n = -1 \text{, i.e. the sample belongs to the rejection class.} \end{cases}$
  - Error function $E(f(x), y_i) = e^{-y_i f(x_i)}$
  - Set of weak learners $h : x \rightarrow \{-1, 1\}$ (these are our filter masks at many different scales)

- **Example of Discrete Two-Class AdaBoost:**
- **For** $t \in \{1...T\}$ **:**
  - Find weak learner h_t that minimizes the weighted sum error of misclassified samples $\epsilon_t = \sum_{i=1, h_t(x_i) \neq y_i}^{n} w_{i,t}$
  - Set $\alpha_t = \frac{1}{2} \ln(\frac{1-\epsilon_t}{\epsilon_t})$ and add the weighted learner to the boosted classifier: $F_t(x) = F_{t-1}(x) + \alpha_t h_t(x)$

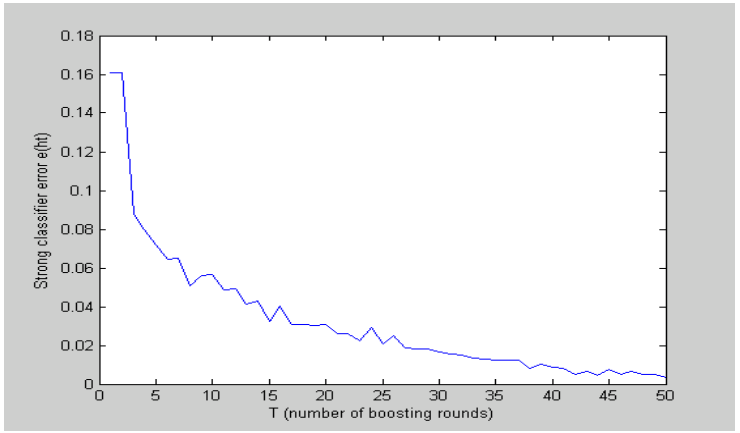


  - Update weights: $w_{i,t+1} = w_{i,t} \begin{cases} e^{-\alpha_t h_t(x_i)} & \text{if } h_t(x_i) = y_i \text{ ➔ correctly classified} \\ e^{\alpha_t h_t(x_i)} & \text{if } h_t(x_i) \neq y_i \text{ ➔ incorrectly classified} \end{cases}$
  - Renormalize $w_{i,t+1}$ for sum 1.

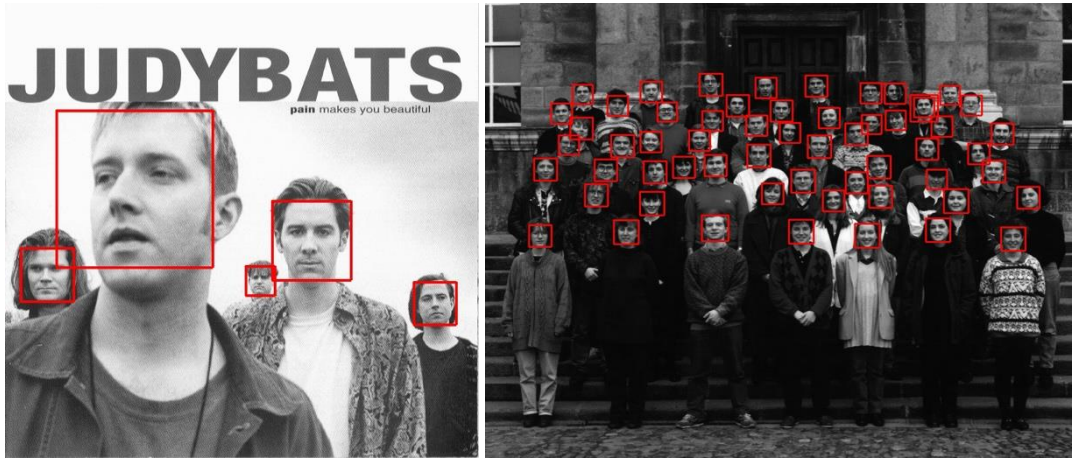
Viola Jones for Face Recognition:



Training Set

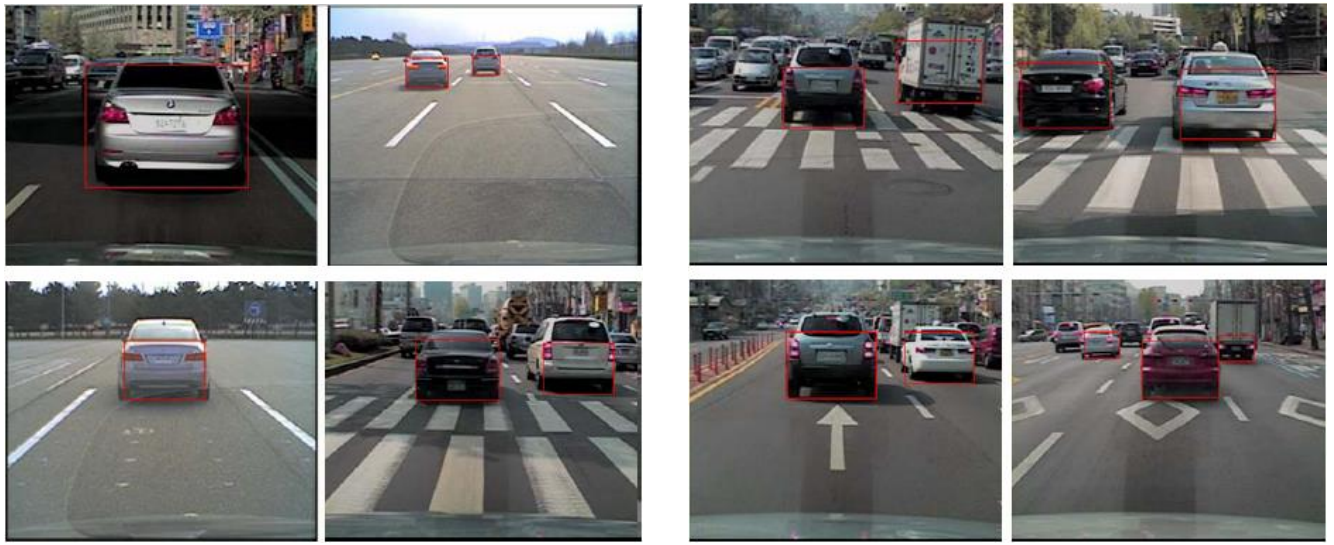


Classification error over number of rounds

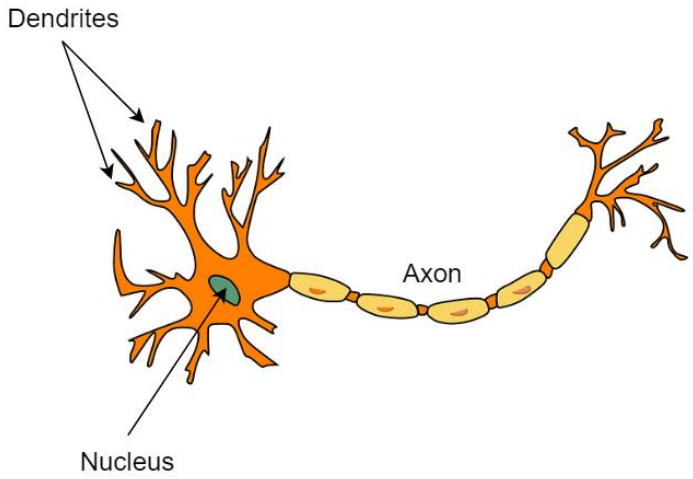


Results

Viola Jones for Vehicle Recognition:

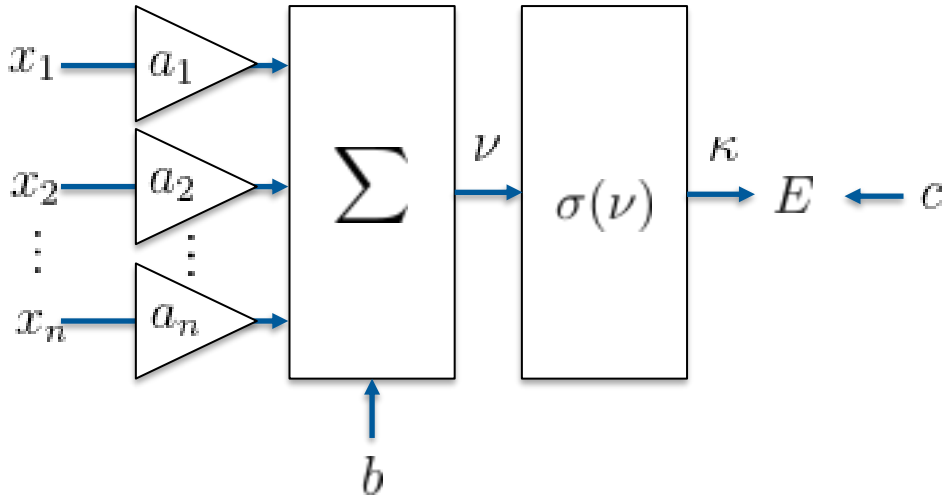


Machine Learning: Neurons Gradient Descent



- **There are many classification algorithms: decision trees, k-nearest neighbor clustering, support vector machines, ...**
- **What is commonly called AI is built on the Artificial Neuron.**
- **It is modelled after a biological neuron, which**
  - collects electrical impulses in its dendrites
  - produces an output in its nucleus and forwards it through its axon,
  - which connects to other neurons‘ dendrites.
- **The AN produces the output**

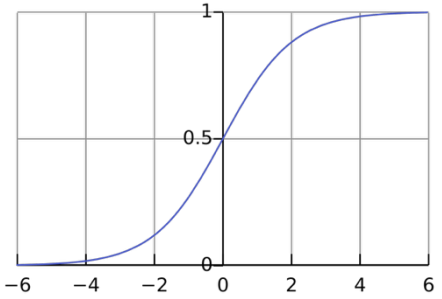
$$\kappa = \sigma(\sum_i x_i a_i + b) = \sigma(\mathbf{x}\mathbf{a}^\top + b)$$



- **In fixed networks, the weights and biases a_i, b form the *model.***
- **σ is the activation function, with several classical options.**

- **One of the most common activation functions is the *sigmoid function***

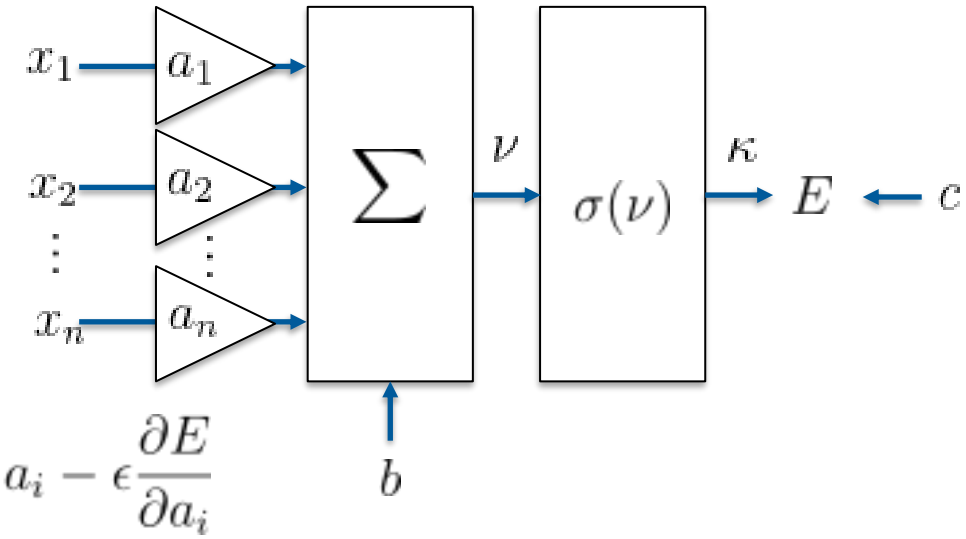
$$\sigma(\nu) = \frac{1}{1 + e^{-\nu}}$$



  It is continuous and differentiable, approaches a binarization function for sufficiently large input values, and its output can be easily interpreted by developers.

- **If we want to train a single neuron, we need to adapt its weights according to the error of its output. We can calculate the absolute error as**

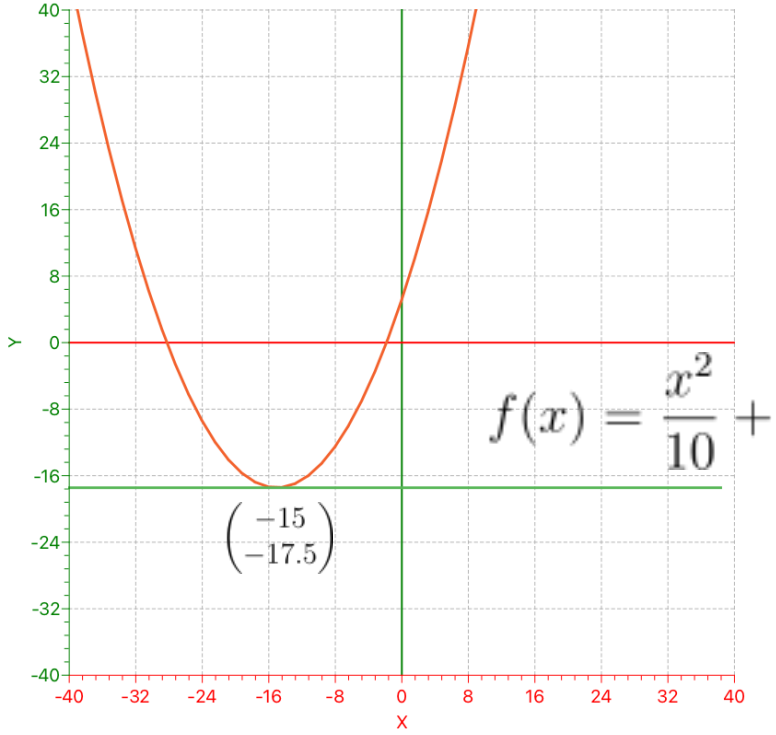
$$E(\mathbf{a}) = \frac{1}{2}(c - \kappa(\mathbf{a}))^2 = \frac{1}{2}(c - \sigma(\mathbf{x}\mathbf{a}^\top + b))^2$$



- **To break this down to the individual weights, we calculate the partial derivatives $\frac{\partial E}{\partial a_i}$ and apply them to the weights incrementally with a scaling factor ϵ :** $a_i' = a_i - \epsilon \frac{\partial E}{\partial a_i}$

- **The scaling factor is usually very small to *descend along the gradient* in the right direction.**

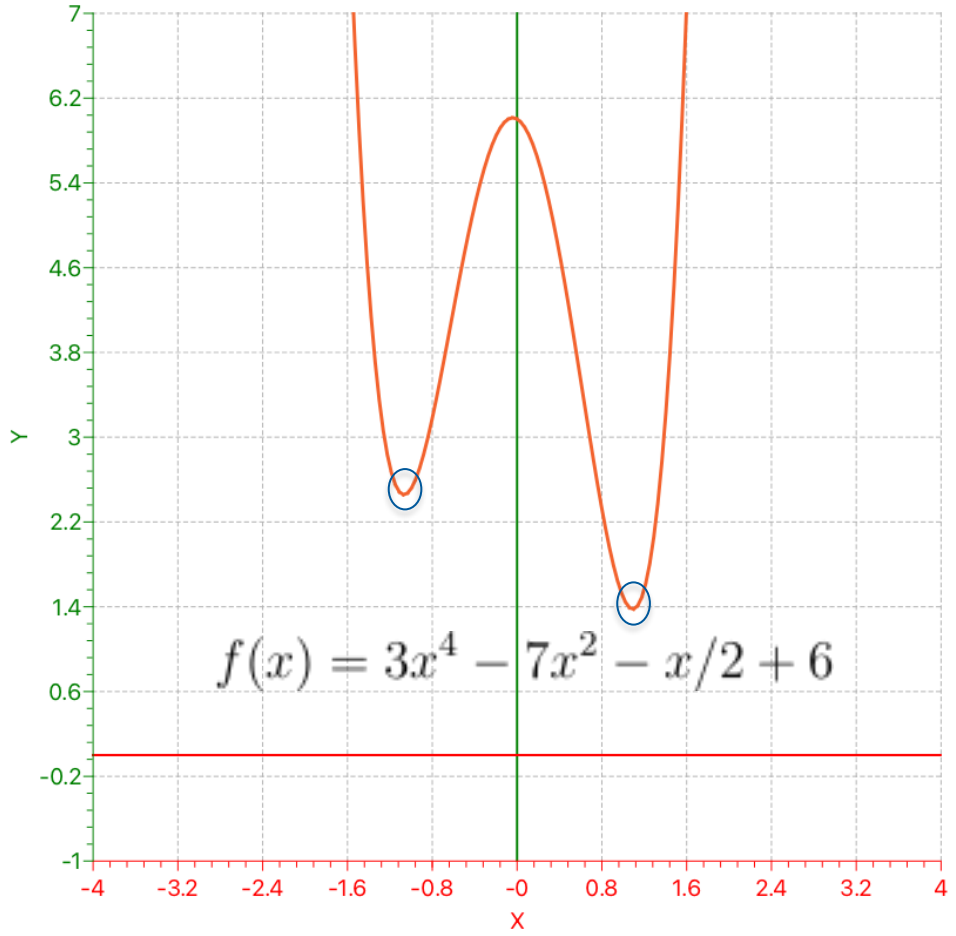
- **Most sensor processing and fusion algorithms are really dealing with optimization.**
- **For problems that have a single optimum, the solution is simple – set derivative to zero and calculate.**



$$f'(x) = \frac{\partial f}{\partial x} = \frac{x}{5} + 3 = 0 \Rightarrow x = -15$$

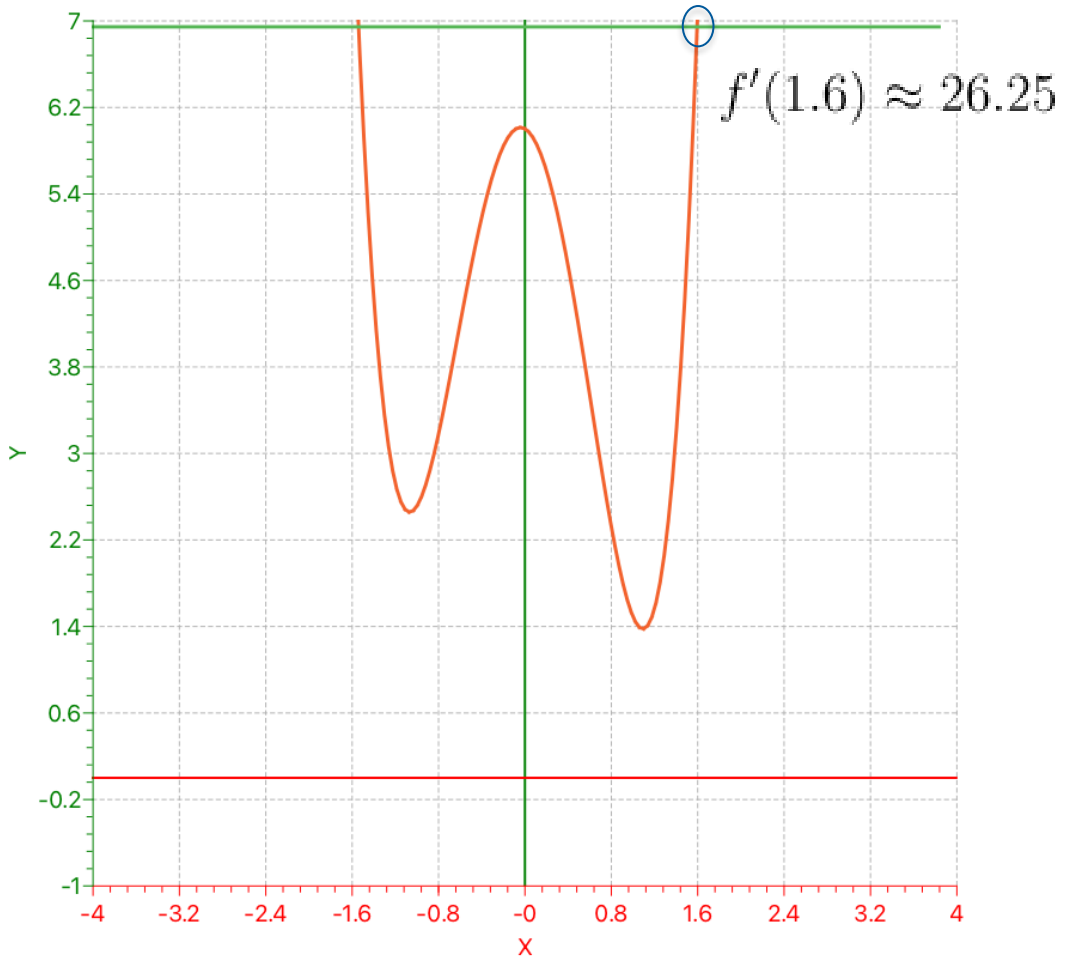
$$y = f(-15) = -17.5$$

- **There often is no optimal solution for a given problem that stays stable over time –**
  - Sensor noise and bias shift over time.
  - Scenarios change, objects appear or vanish.
  - Multiple hypotheses may look similar, sometimes choosing the absolute optimum is misleading.
- **In many cases trying to find the global optimum is simply too costly.**
- **Even for the upcoming simple example of a Neural Net, we will be looking at ~14,000 parameters.**



$$f(x) = 3x^4 - 7x^2 - x/2 + 6$$
$$f'(x) = 12x^3 - 14x - 1/2$$

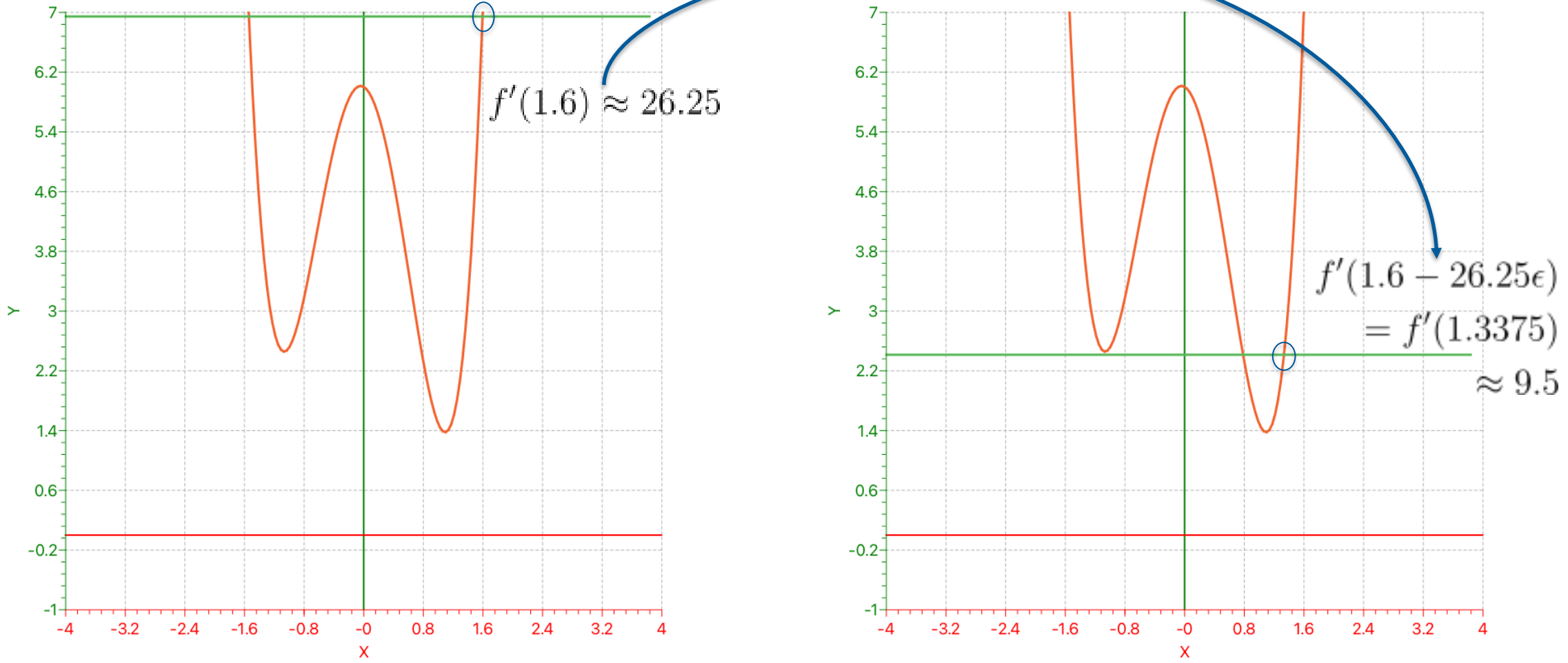
- **We solve this with an iterative process called *Gradient Descent*.**
  - Start at a randomly determined initial position.
  - Calculate the gradient of the function at this position.
    (For the 1-dimensional case, this is the derivative.)
  - Move the solution point slightly in negative direction of the gradient, proportional to the gradient value.

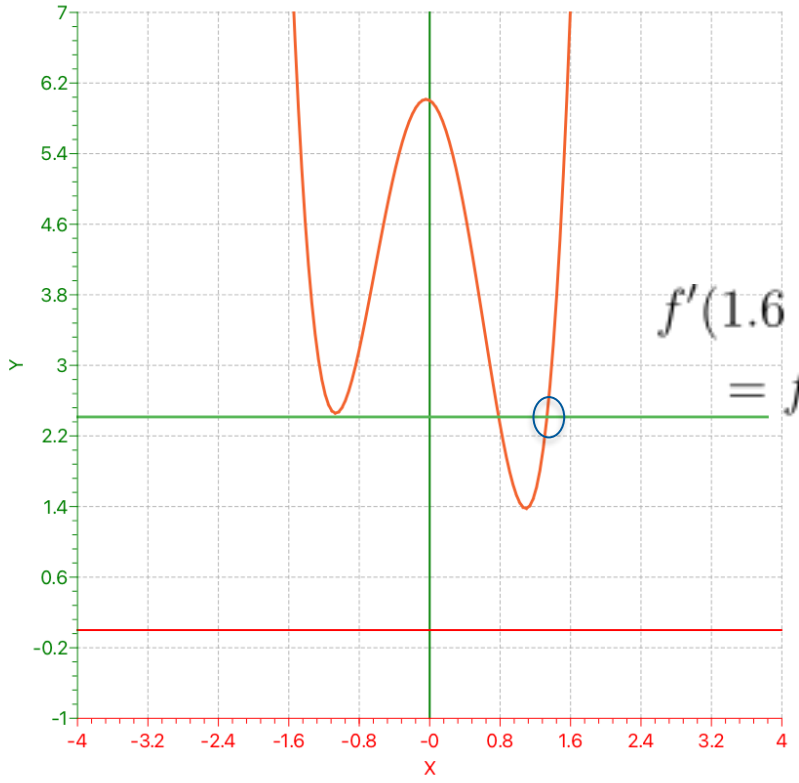


Gradient Descent

$$f(x) = 3x^4 - 7x^2 - x/2 + 6$$

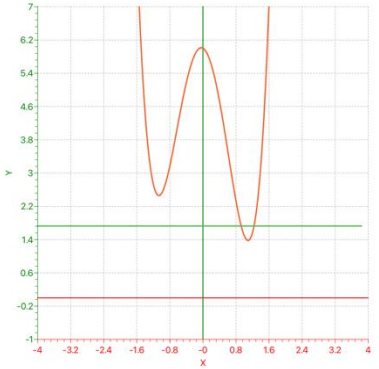
$$f'(x) = 12x^3 - 14x - 1/2$$



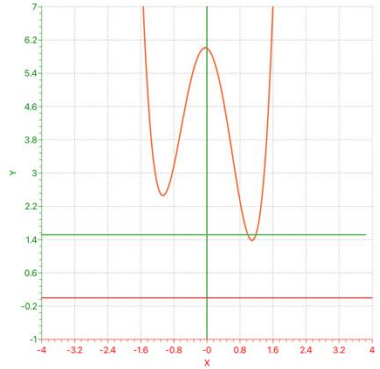


$$f'(1.6 - 26.25\epsilon) = f'(1.3375) \approx 9.5$$

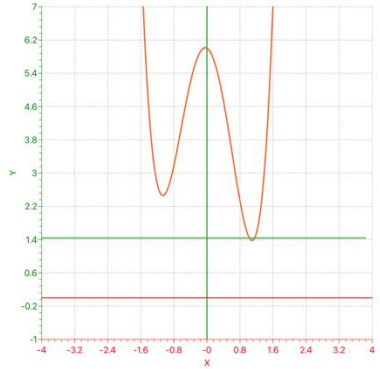
$$f'(1.3375 - 9.5\epsilon) = f'(1.2425) \approx 5.12$$



$$f'(1.1425 - 5.12\epsilon) = f'(1.1913) \approx 3.11$$



$$f'(1.1425 - 3.11\epsilon) = f'(1.1602) \approx 2$$



$$f'(1.14) \approx 1.32$$
$$f'(1.127) \approx 0.99$$
$$f'(1.117) \approx 0.58$$
$$f'(1.111) \approx 0.4$$
$$\vdots$$
$$f'(1.09756) \approx 0$$
$$f(1.09756) \approx 1.3722$$

We arrive at a local minimum of 1.3722 at position x=1.09756.

For the multi-variable case $f(x_1, x_2, \ldots, x_n)$**,**

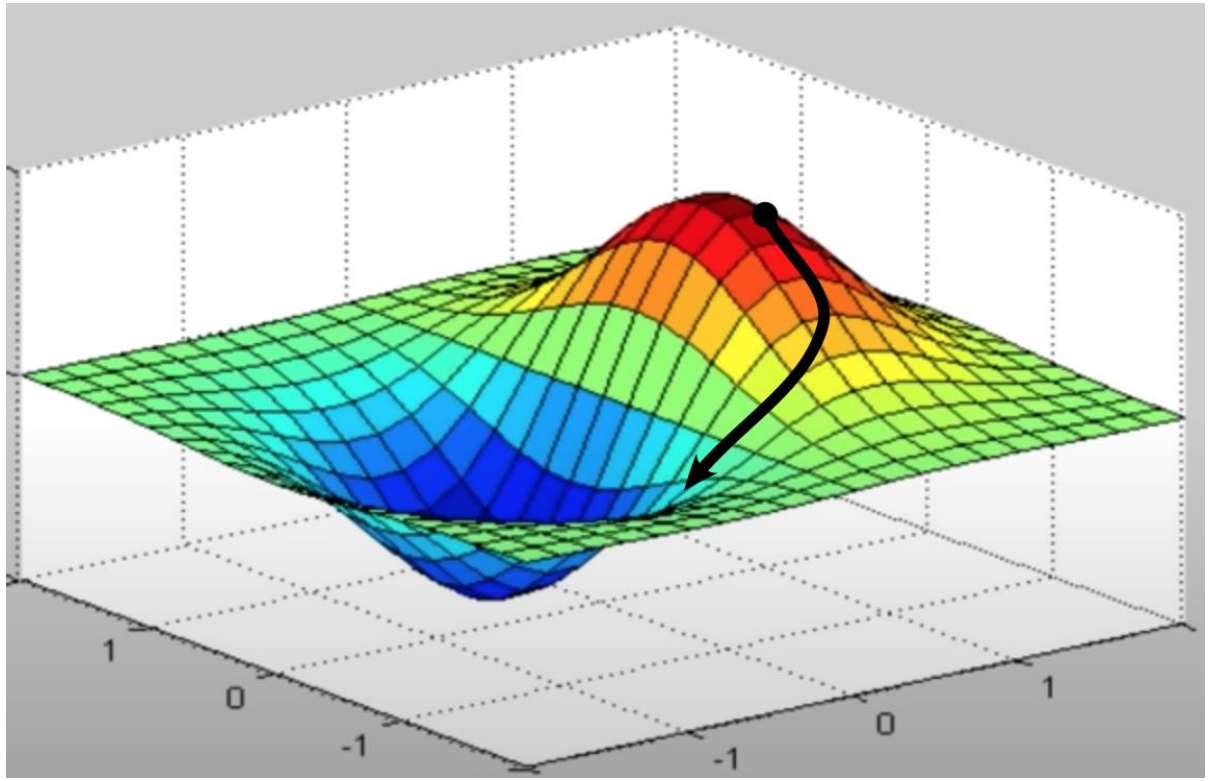
we need the *gradient vector,* i.e. the vector of partial derivatives

$$f'(x_1, x_2, \ldots, x_n) = \nabla f = \begin{pmatrix} \partial f / \partial x_1 \\ \partial f / \partial x_2 \\ \vdots \\ \partial f / \partial x_n \end{pmatrix}$$

We start at a random point in *n*-dimensional space and calculate ∇f **.**

We need to *descend in the direction of* $-\nabla f$ **.**

The *length of* $-\nabla f$ **gives us an indication of how much we need to descend.**

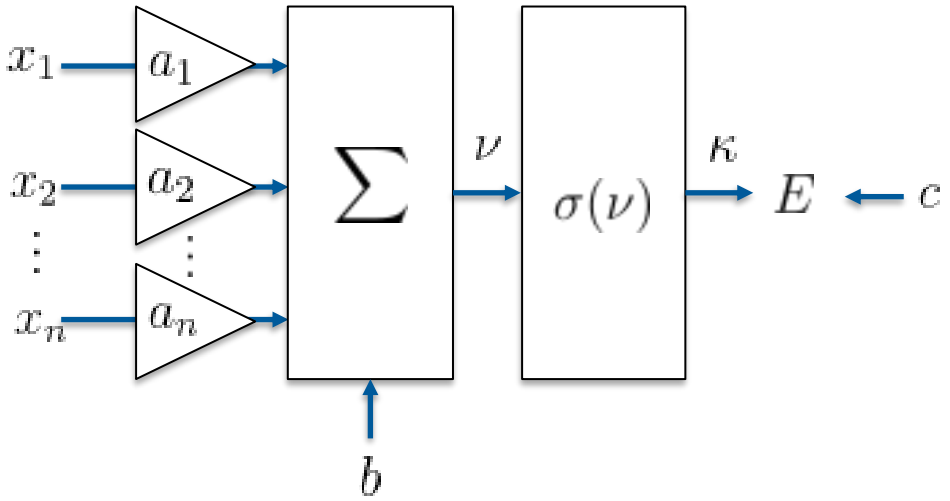


- **Let‘s look at a single neuron.**
- **We want to find the *minimum of the error function***

$$E(\mathbf{a}) = \frac{1}{2}(c - \kappa(\mathbf{a}))^2 = \frac{1}{2}(c - \sigma(\mathbf{x}\mathbf{a}^\top + b))^2$$

using gradient descent. So we need the gradient vector

$$\nabla E = \begin{pmatrix} \partial E/\partial a_1 \\ \partial E/\partial a_2 \\ \vdots \\ \partial E/\partial a_n \end{pmatrix}$$



- **Applying the chain rule twice, the individual components of the gradient vector are**

$$\frac{\partial E}{\partial a_i} = \frac{\partial E}{\partial \kappa}\frac{\partial \kappa}{\partial a_i} = \frac{\partial E}{\partial \kappa}\frac{\partial \kappa}{\partial \nu}\frac{\partial \nu}{\partial a_i}$$

- **For** $E(\mathbf{a}) = \frac{1}{2}(c - \kappa(\mathbf{a}))^2 = \frac{1}{2}(c - \sigma(\mathbf{x}\mathbf{a}^\top + b))^2$,

  find $\frac{\partial E}{\partial a_i} = \frac{\partial E}{\partial \kappa}\frac{\partial \kappa}{\partial a_i} = \frac{\partial E}{\partial \kappa}\frac{\partial \kappa}{\partial \nu}\frac{\partial \nu}{\partial a_i}$.

- **The first one is simple:** $\frac{\partial E}{\partial \kappa} = \frac{\partial}{\partial \kappa}\frac{1}{2}(c - \kappa)^2 = \frac{\partial}{\partial \kappa}\frac{1}{2}(c^2 - 2c\kappa + \kappa^2) = \frac{1}{2}(-2c + 2\kappa) = \kappa - c$

- **The last one is also simple:** $\frac{\partial \nu}{\partial a_i} = \frac{\partial}{\partial a_i}(\sum_i x_i a_i + b) = x_i$

  and $\frac{\partial \nu}{\partial b} = \frac{\partial}{\partial b}(\sum_i x_i a_i + b) = 1$ (we'll skip the b computation for the rest of the discussion – it's the same as the a_i without the last term.)

- **We still need to solve** $\frac{\partial \kappa}{\partial \nu}$ **. This is dependent on the activation function. Let's look at the sigmoid.**

- **Derivative of the Sigmoid** $\sigma(x) = \frac{1}{1+e^{-x}} = \frac{e^x}{1+e^x}$

- **With the chain rule** $f(x) = u(v(x)) \Rightarrow f'(x) = u'(v(x))v'(x)$ **and** $v(x) = 1 + e^x$**:**

$$
\begin{aligned}
f'(x) &= \frac{e^x(1+e^x) - e^x e^x}{(1+e^x)^2} \\
&= \frac{e^x}{(1+e^x)^2} \\
&= \left(\frac{e^x}{1+e^x}\right)\left(\frac{1}{1+e^x}\right) \\
&= \left(\frac{e^x}{1+e^x}\right)\left(\frac{1+e^x-e^x}{1+e^x}\right) \\
&= \left(\frac{e^x}{1+e^x}\right)\left(1-\frac{e^x}{1+e^x}\right) \\
&= f(x)(1-f(x))
\end{aligned}
$$

- **So with the sigmoid function,**

$$\frac{\partial E}{\partial a_i} = \frac{\partial E}{\partial \kappa}\frac{\partial \kappa}{\partial a_i} = \frac{\partial E}{\partial \kappa}\frac{\partial \kappa}{\partial \nu}\frac{\partial \nu}{\partial a_i}$$

becomes

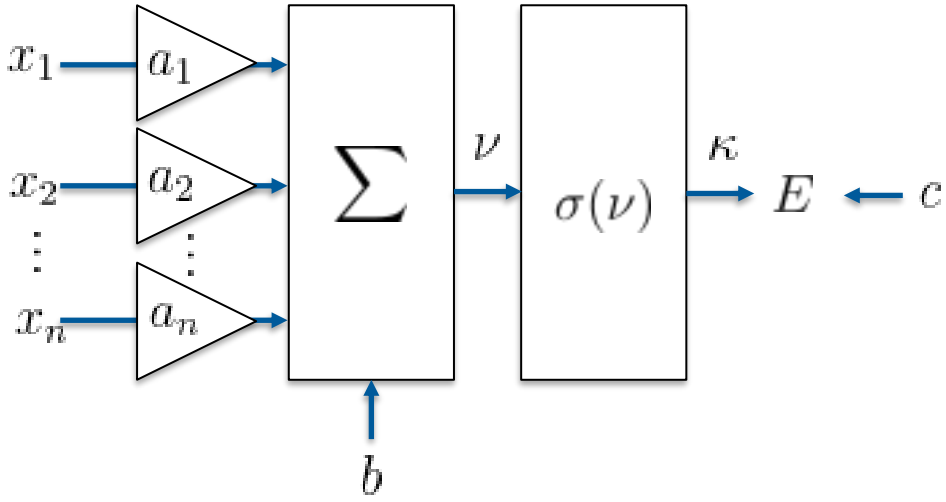
$$(\kappa - c)\sigma(a_i)(1 - \sigma(a_i))x_i$$

- **Constructing the gradient vector** $\nabla E = \begin{pmatrix} \partial E/\partial a_1 \\ \partial E/\partial a_2 \\ \vdots \\ \partial E/\partial a_n \end{pmatrix}$**, we can now „wiggle our way down“**

using $a_i' = a_i - \epsilon\frac{\partial E}{\partial a_i}$ **or** $\mathbf{a}' = \mathbf{a} - \epsilon\nabla E$ **.**

- **If we want to train a single neuron, we need to adapt its weights according to the error of its output. We can calculate the absolute error as**

$$E(\mathbf{a}) = \frac{1}{2}(c - \kappa(\mathbf{a}))^2 = \frac{1}{2}(c - \sigma(\mathbf{x}\mathbf{a}^\top + b))^2$$



- **To break this down to the individual weights, we calculate the partial derivatives $\frac{\partial E}{\partial a_i}$ and apply them to the weights incrementally with a scaling factor ϵ:**

$$a_i' = a_i + \epsilon \frac{\partial E}{\partial a_i}$$

- **The scaling factor is usually very small to *descend along the gradient* in the right direction.**



Technische Hochschule Ingolstadt

Fakultät Elektro- und Informationstechnik

Thank you very much!